Computer Architecture and Operating Systems
COSC 22073/BECS 22233

Lesson 06: Processor Design

Department of Statistics & Computer Science
Faculty of Science
University of Kelaniya

What is the Processor Design?

- Processor design refers to the process of creating the architecture, microarchitecture, and physical implementation of a central processing unit (CPU) or processor for a computer or digital device. It involves a combination of hardware and software considerations and encompasses various stages to produce an efficient and functional processor.

What is the aim of the Processor Design?

- The aim of processor design is to create a device that can perform computing tasks quickly, efficiently, and securely while also being cost-effective and flexible.

The Main Aims of Processor Design

- **Fast**: A processor should be able to execute instructions quickly to allow for high-performance computing.
- **Efficient**: A processor should be able to execute instructions with a minimum of power consumption, heat generation, and other overhead to allow for long battery life, reduced cooling requirements, and increased reliability.
- **Scalable**: A processor should be designed in a way that allows it to be used in a wide range of systems, from low-end devices to high-performance computing systems.

The Main Aims of Processor Design

- **Cost-effective**: A processor should be designed to meet specific cost targets, such as low production costs, low power consumption, and low heat generation, to make it affordable for a wide range of applications.
- **Flexible**: A processor should be designed to allow for easy customization and integration into different systems to support a wide range of applications and use cases.
- **Secure**: A processor should be designed with security in mind to prevent unauthorized access, data theft, and other types of security attacks. are to create a device that is

Basic Approach of the Processor Design

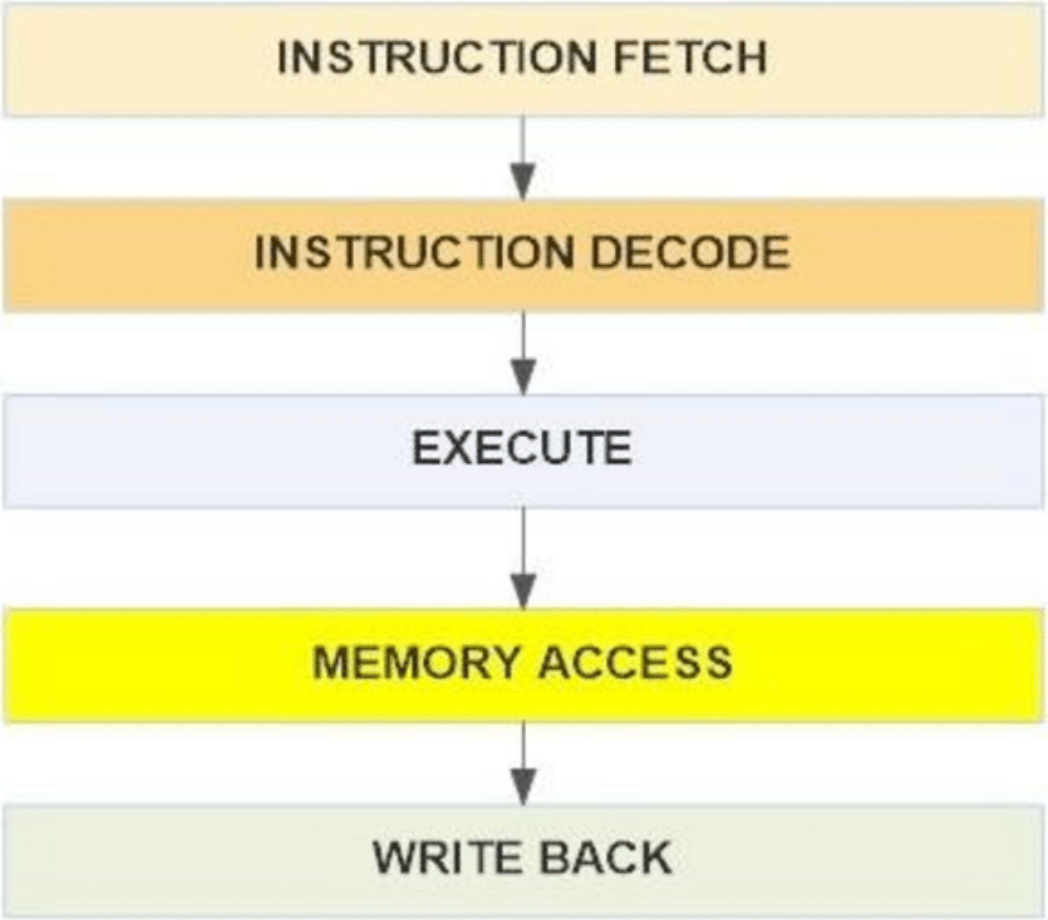
- Divide the processing into stages
- Design each stage separately

Example: A Car Assembly Line

Similar to a car assembly line

- Cast raw metal into the chassis of a car
- Build the Engine
- Assemble the engine and the chassis
- Place the dashboard, and upholster

Processor Stages



Components of a Processor

Processors consist of two main components:

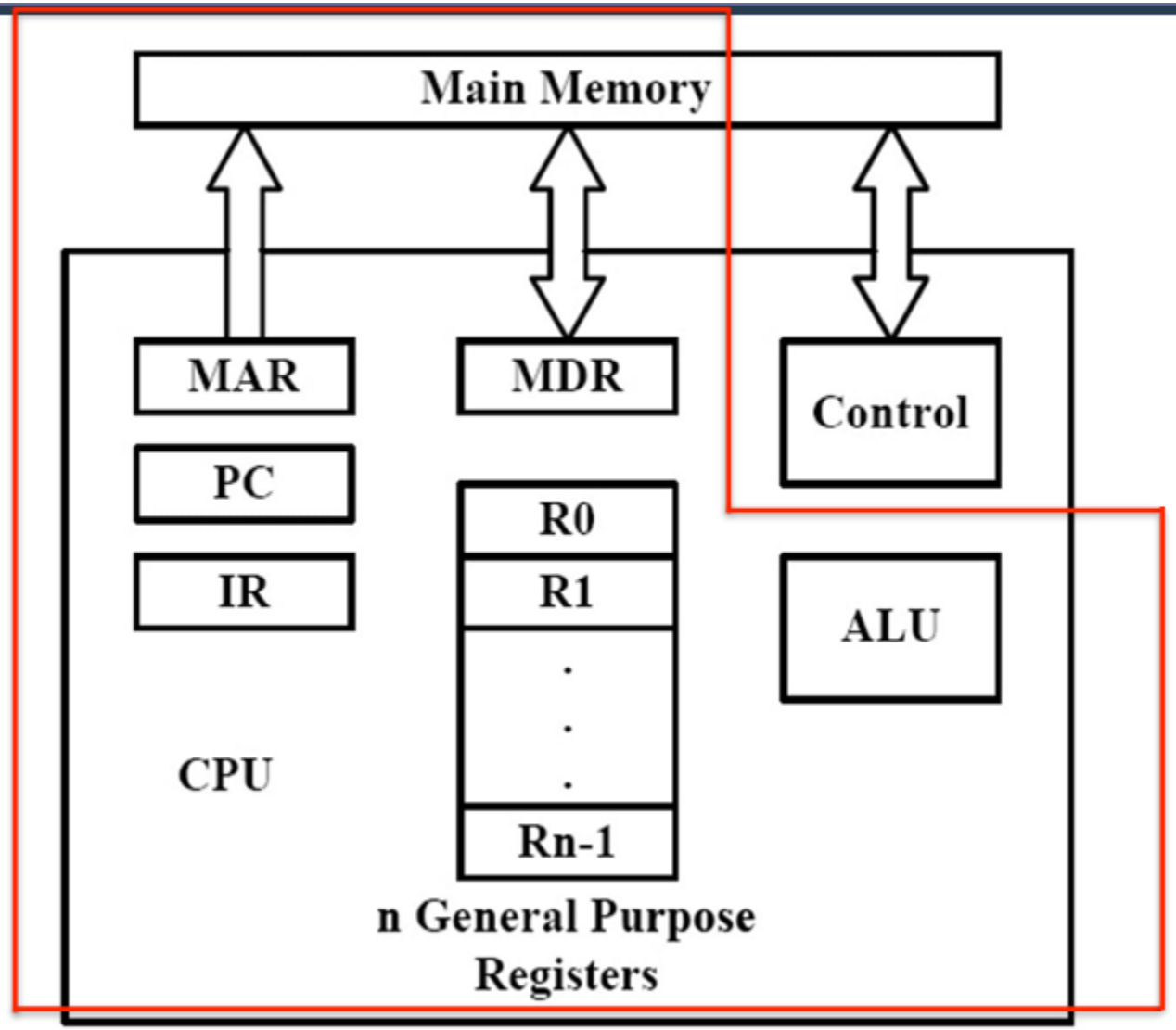
- Controller

whose role is to generate the appropriate signals to control the movement of instructions, and data in the data path.

- Datapath

The data path consists of all the elements in a processor that are dedicated to storing, retrieving, and processing data such as register files, memory, and the ALU.

Controller and the Data Path



Building Blocks of a Processor

Two types of functional units

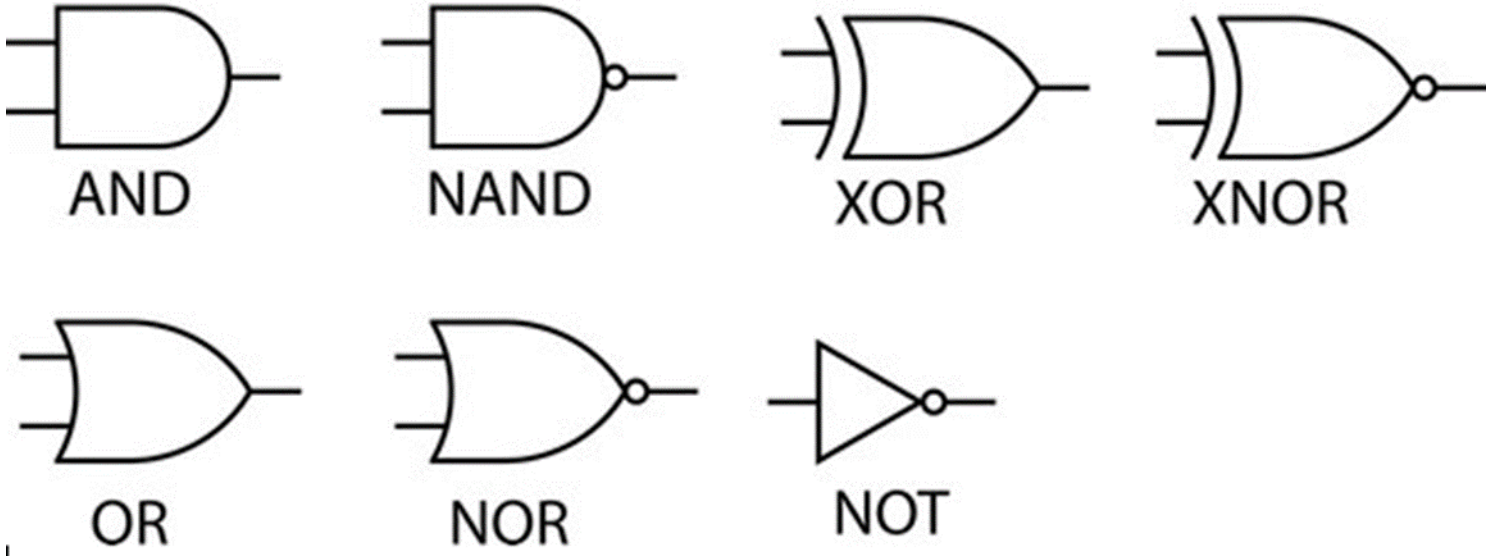
- Elements that operates on data values (Combinational).
  - Output is function of current input
  - No memory
- Elements that contains state (Sequential)
  - Output is function of current and previous inputs.
  - State = Memory

Combinational Circuit Examples

- Gates: and, or, nand, nor, xor, inverter
- Multiplexer
- Decoder
- Adder, Subtractor, Comparator
- ALU

Combinational Circuit Examples

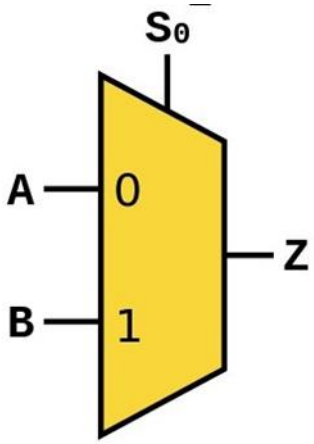
Gates: and, or, nand, nor, xor, inverter (not)

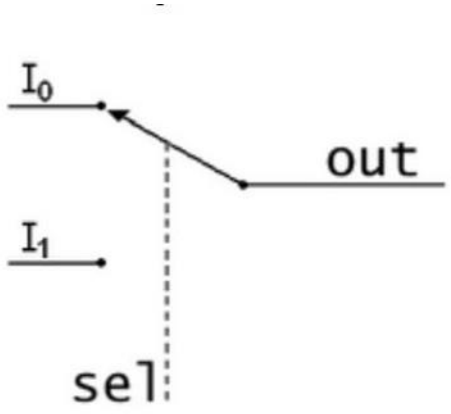


Combinational Circuit Examples

Multiplexers: It is simply a Data Selector

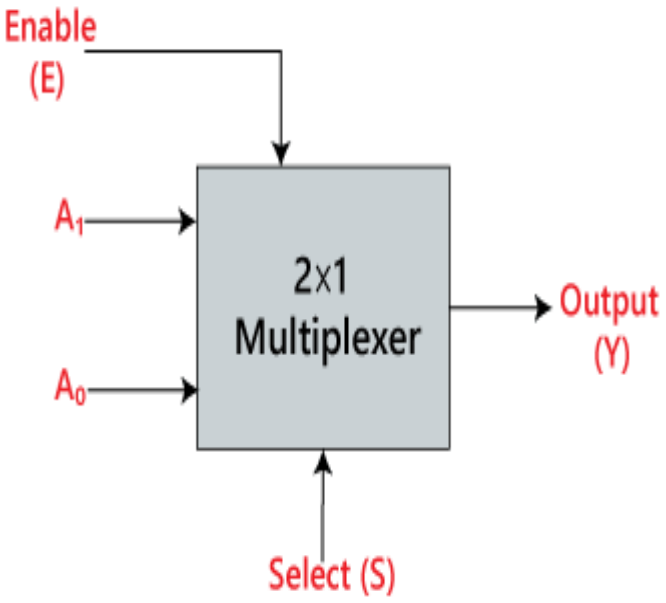
Advantages: Reduces no of wires, complexity and cost.





Combinational Circuit Examples

Block Diagram



Truth Table

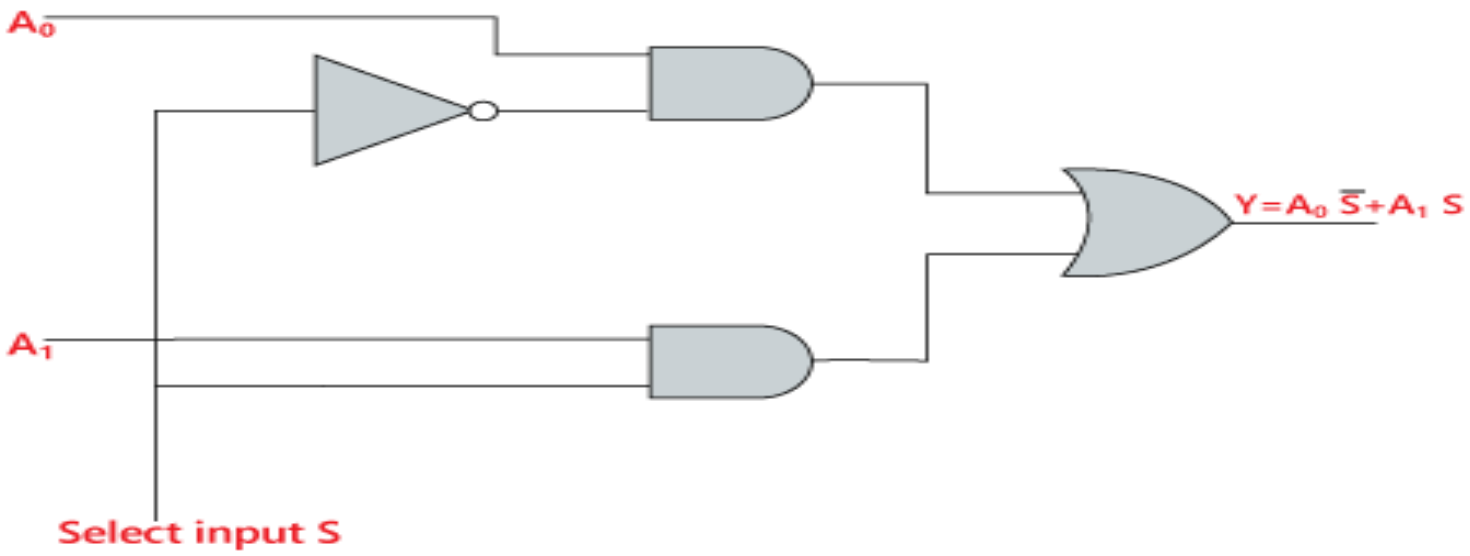
INPUTS	Output
S_0	Y
0	A_0
1	A_1

The logical expression of the term Y is as follows:
$Y=S_0'.A_0+S_0.A_1$

Combinational Circuit Examples

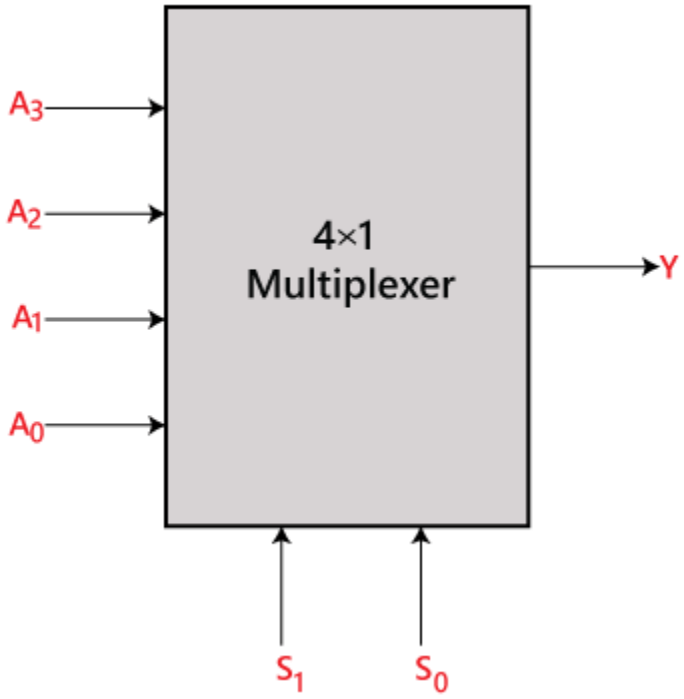
Logical circuit of the 2*1 Multiplexer.

$Y=S_0'.A_0+S_0.A_1$



Combinational Circuit Examples

Block Diagram



Truth Table

INPUTS		Output
S_1	S_0	Y
0	0	A_0
0	1	A_1
1	0	A_2
1	1	A_3

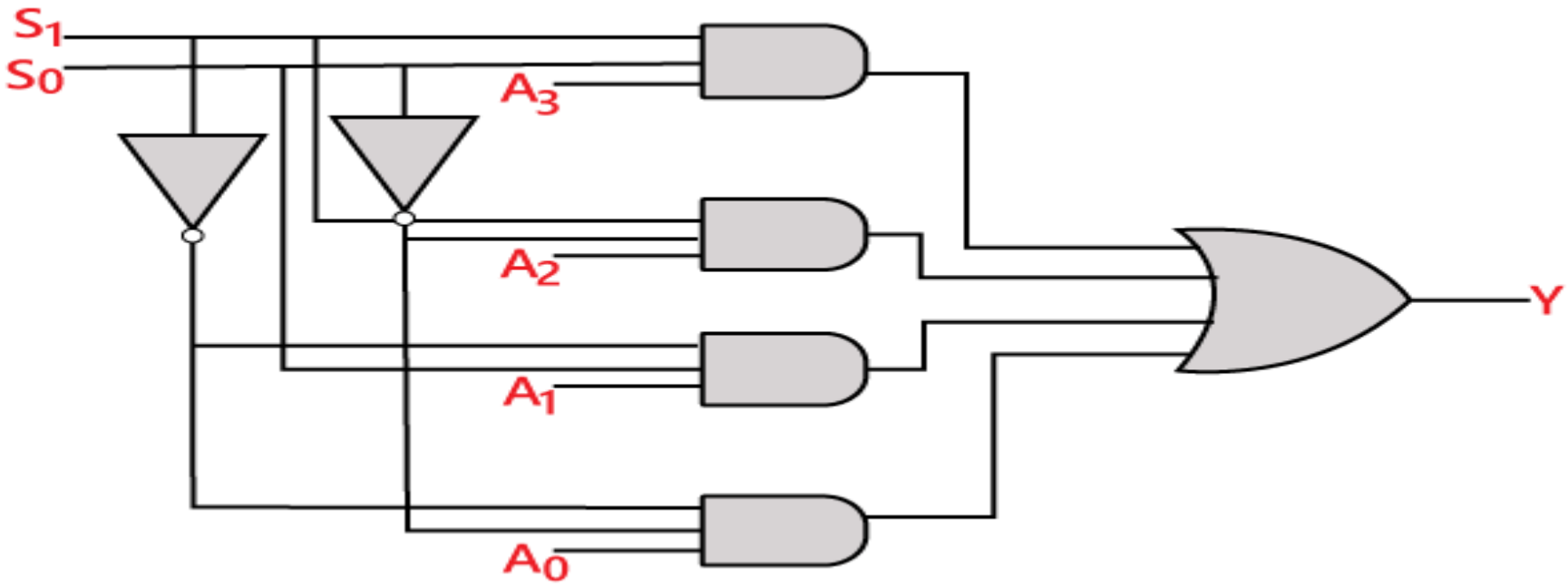
The logical expression of the term Y is as follows:

$Y=S_1' S_0' A_0+S_1' S_0 A_1+S_1 S_0' A_2+S_1 S_0 A_3$

Combinational Circuit Examples

Logical circuit of the 4*1 Multiplexer.

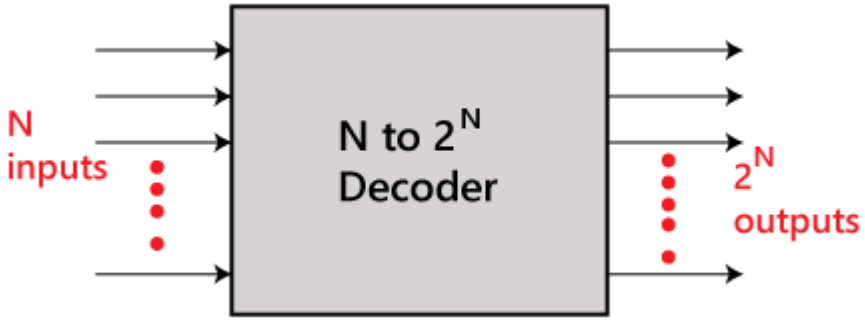
- $Y=S_1' S_0' A_0+S_1' S_0 A_1+S_1 S_0' A_2+S_1 S_0 A_3$



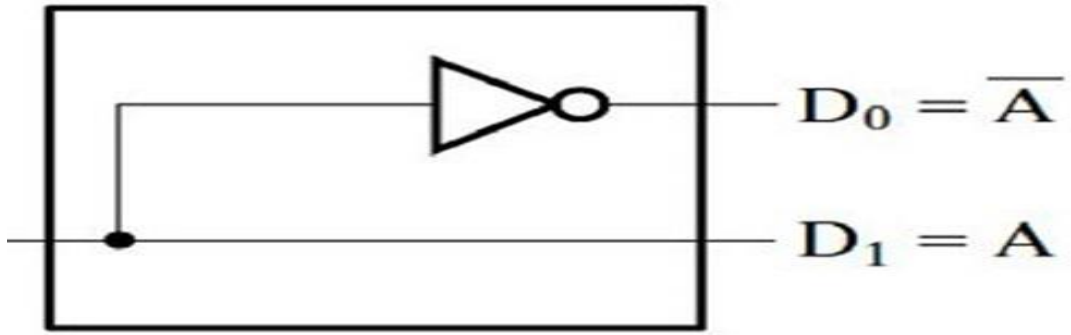
Combinational Circuit Examples

Decoder:

Input	Output	
A	D_0	D_1
0	1	0
1	0	1



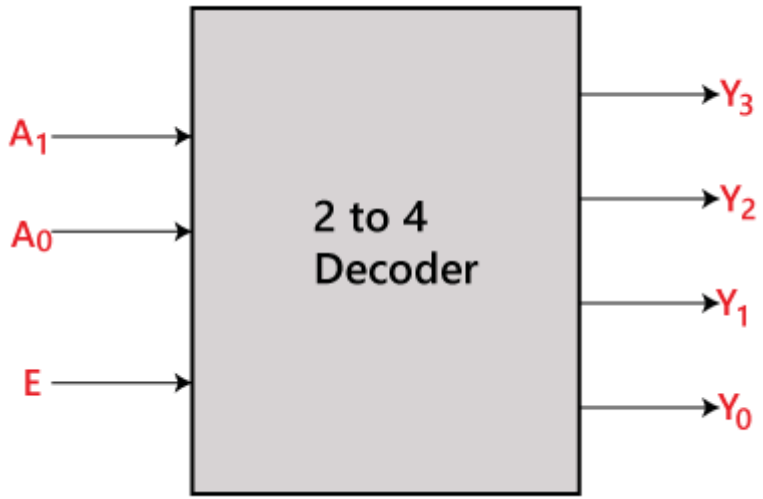
Decoder Block
(High level Diagram)



Detailed Logic Diagram

Combinational Circuit Examples

Block Diagram



Truth Table

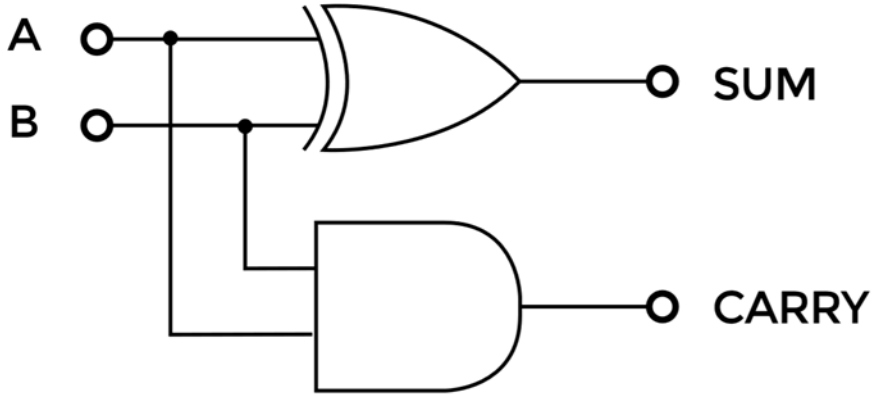
Enable	INPUTS		OUTPUTS			
E	A_1	A_0	Y_3	Y_2	Y_1	Y_0
0	X	X	0	0	0	0
1	0	0	0	0	0	1
1	0	1	0	0	1	0
1	1	0	0	1	0	0
1	1	1	1	0	0	0

The logical expression of the term Y0, Y0, Y2, and Y3 is as follows:

$$Y_3=E.A_1.A_0$$
$$Y_2=E.A_1.A_0'$$
$$Y_1=E.A_1'.A_0$$
$$Y0=E.A_1'.A_0'$$

Combinational Circuit Examples

Adders: Half Adders

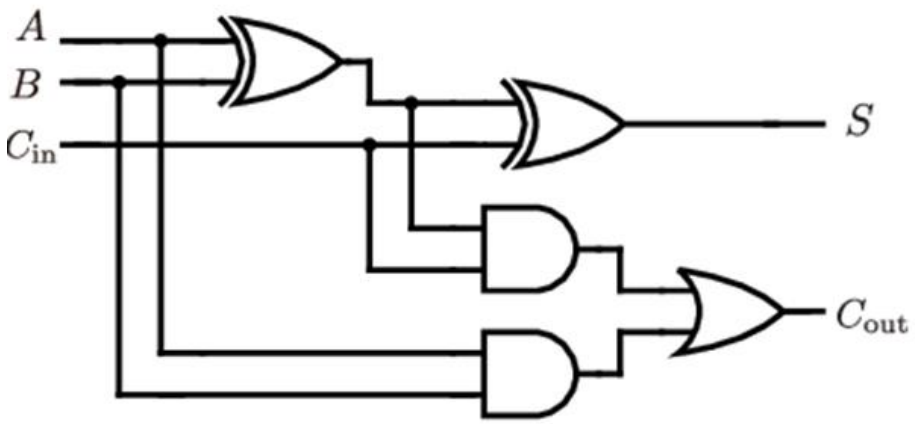


A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Can only add two single bit binary numbers at a time

Combinational Circuit Examples

Adders: Full Adders



Inputs			Outputs	
A	B	C_{in}	S	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Example :

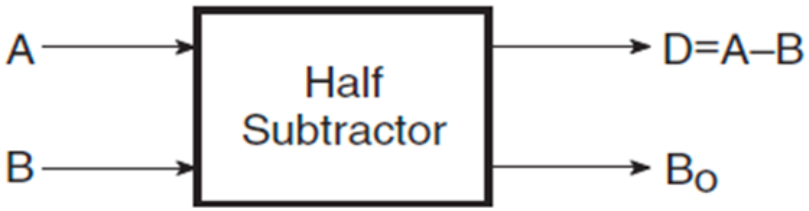
1011 + 0111 = ?

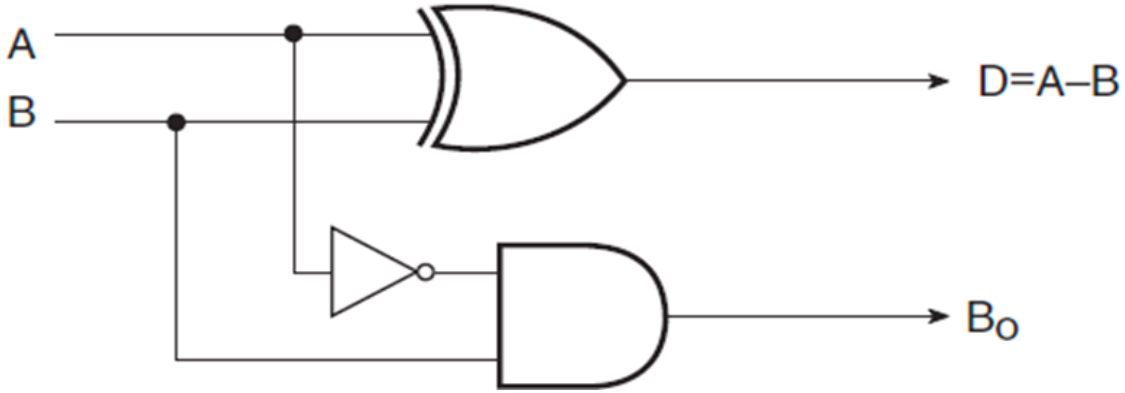
Bit postion	A	B	Cin	Sum	Cout
LSB	1	1	0	0	1
1	1	1	1	1	1
2	0	1	1	0	1
MSB	1	0	1	0	1

Combinational Circuit Examples

Subtractors: Half Subtractor

A	B	D	B_O
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

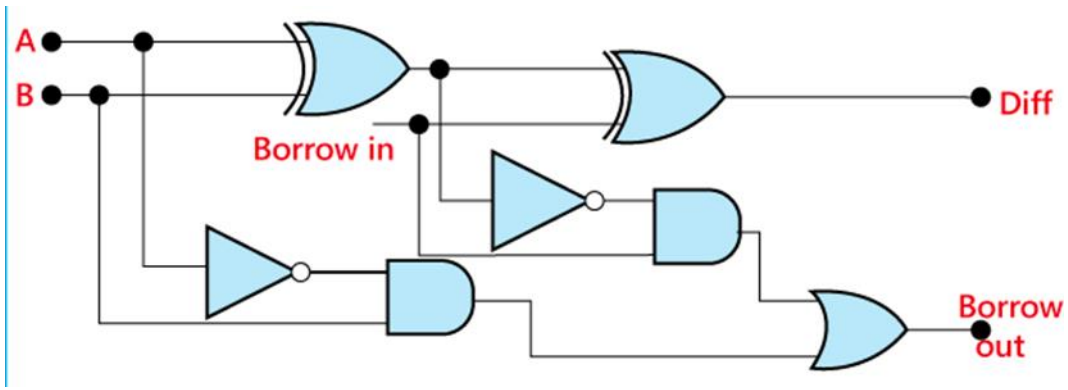




Can only substract two binary digits without considering previous borrows.

Combinational Circuit Examples

Subtractors: Full Subtractor



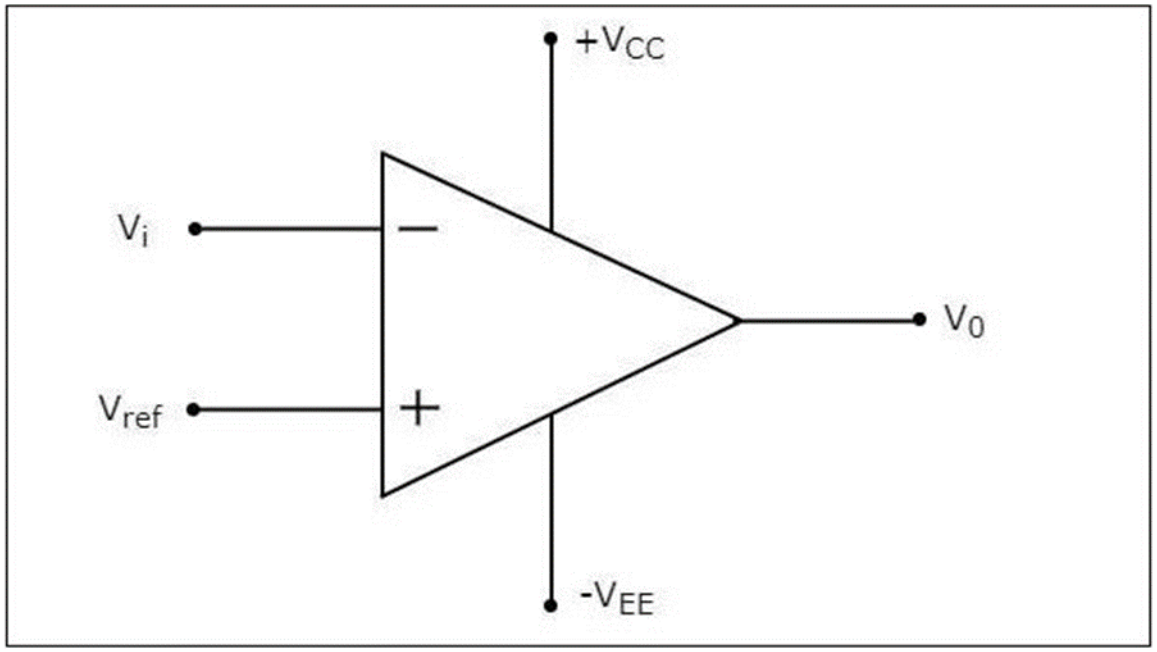
Example 1011-0111

Bit Position	A	B	Borrow in	Difference	Borrow out
LSB	1	1	0	0	0
1	1	1	0	0	0
2	0	1	0	1	1
MSB	1	0	1	0	0

A	B	B_{in}	D	B_{out}
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

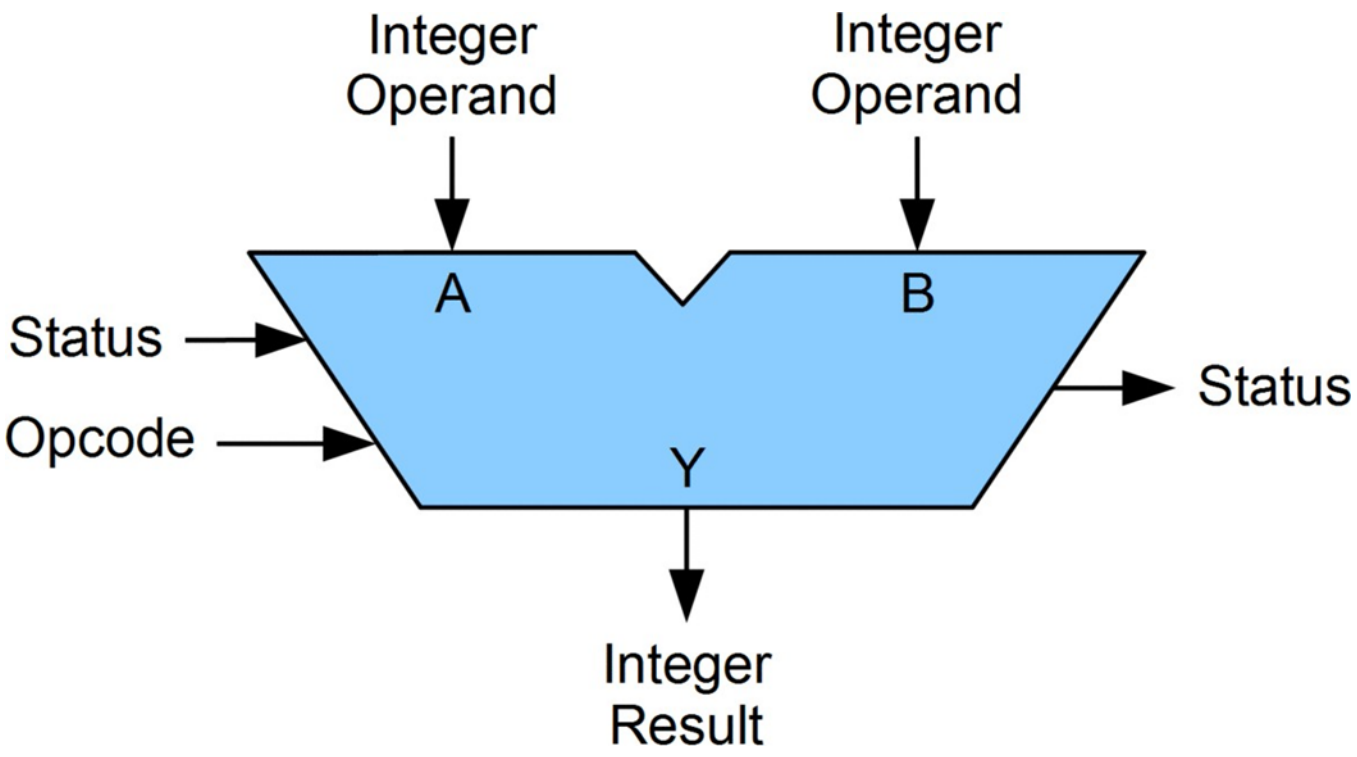
Combinational Circuit Examples

Comparator:



Combinational Circuit Examples

ALU:

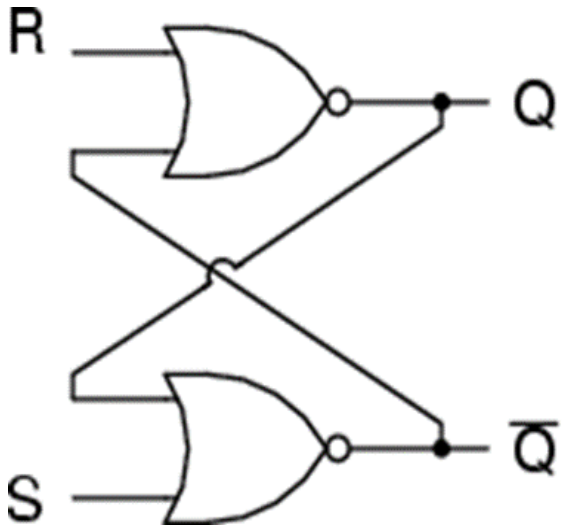


Sequential Circuit Examples

- Flip-flops
- Counters
- Registers
- Register files
- Memories.

Sequential Circuit Examples

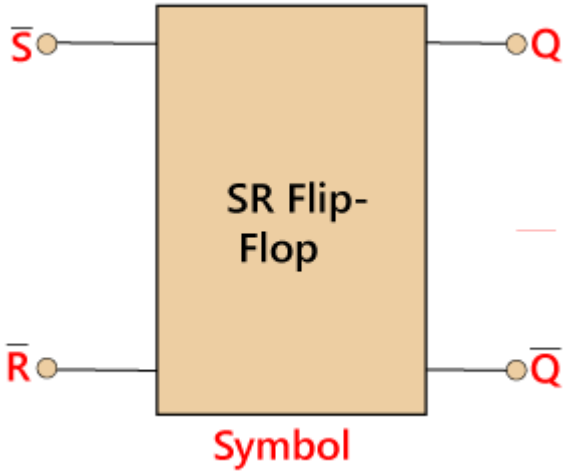
S-R Latch

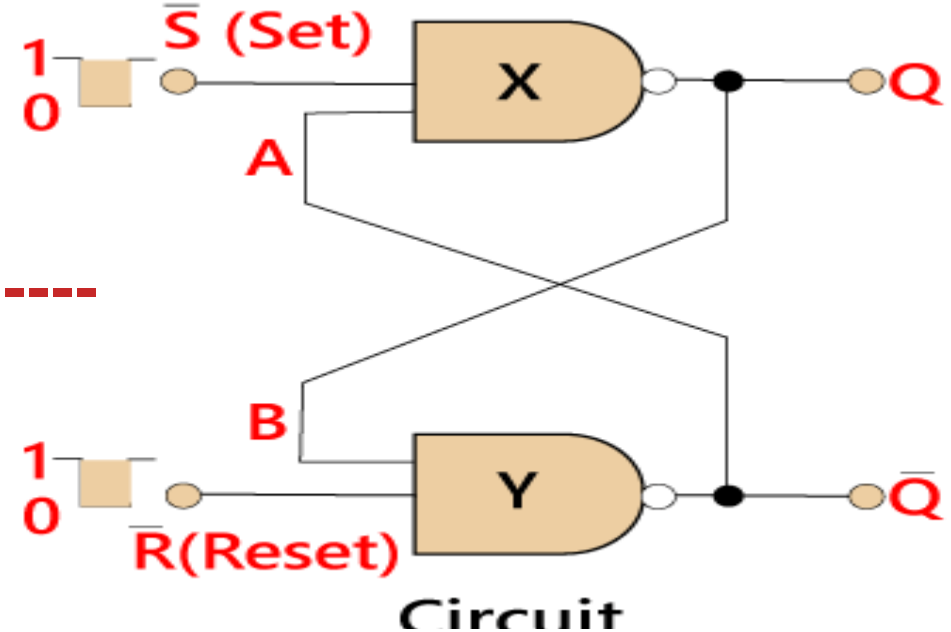


S	R	Q	$\overline{Q}$
0	0	latch	latch
0	1	0	1
1	0	1	0
1	1	0	0

Sequential Circuit Examples

S-R Latch with NAND

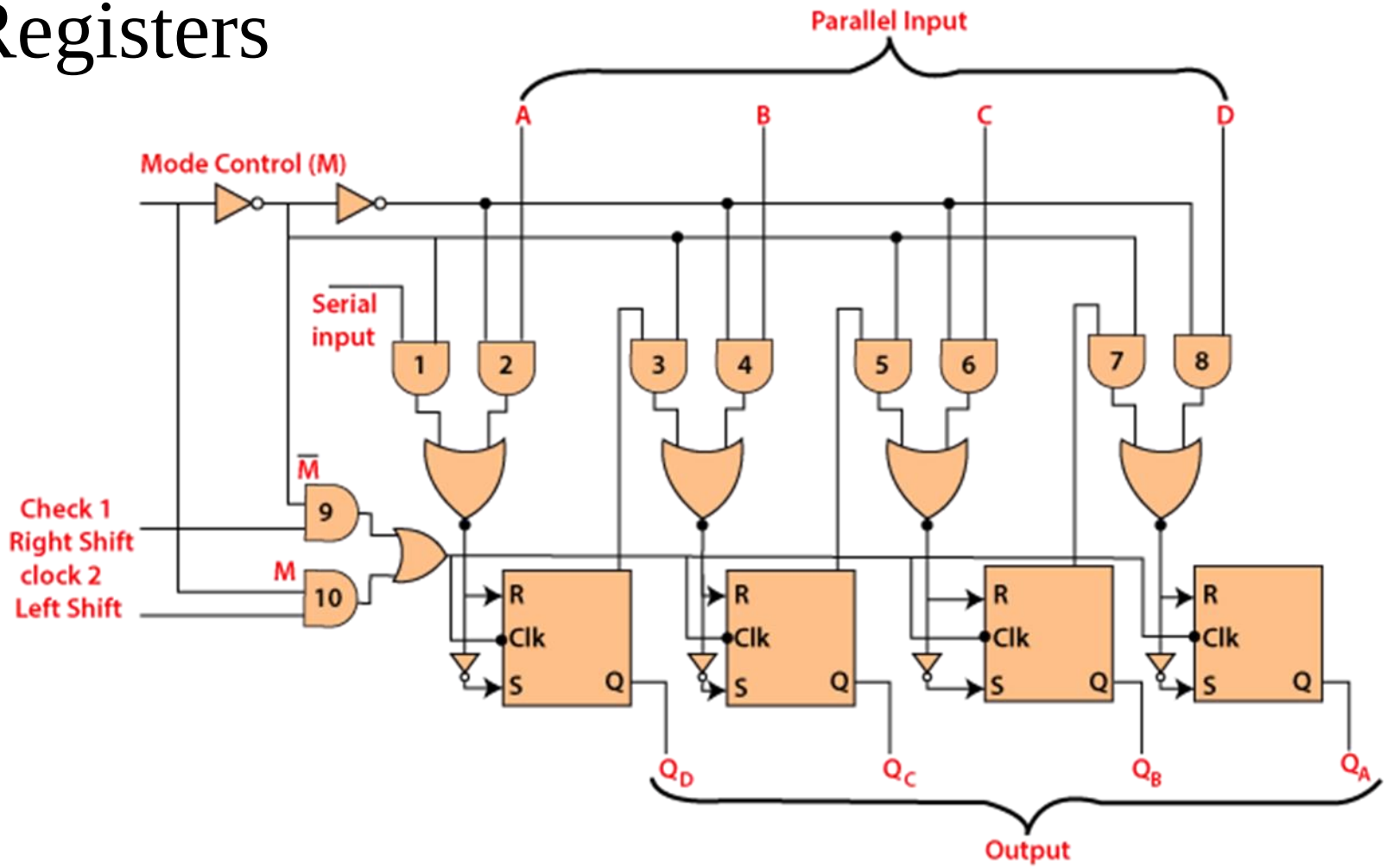




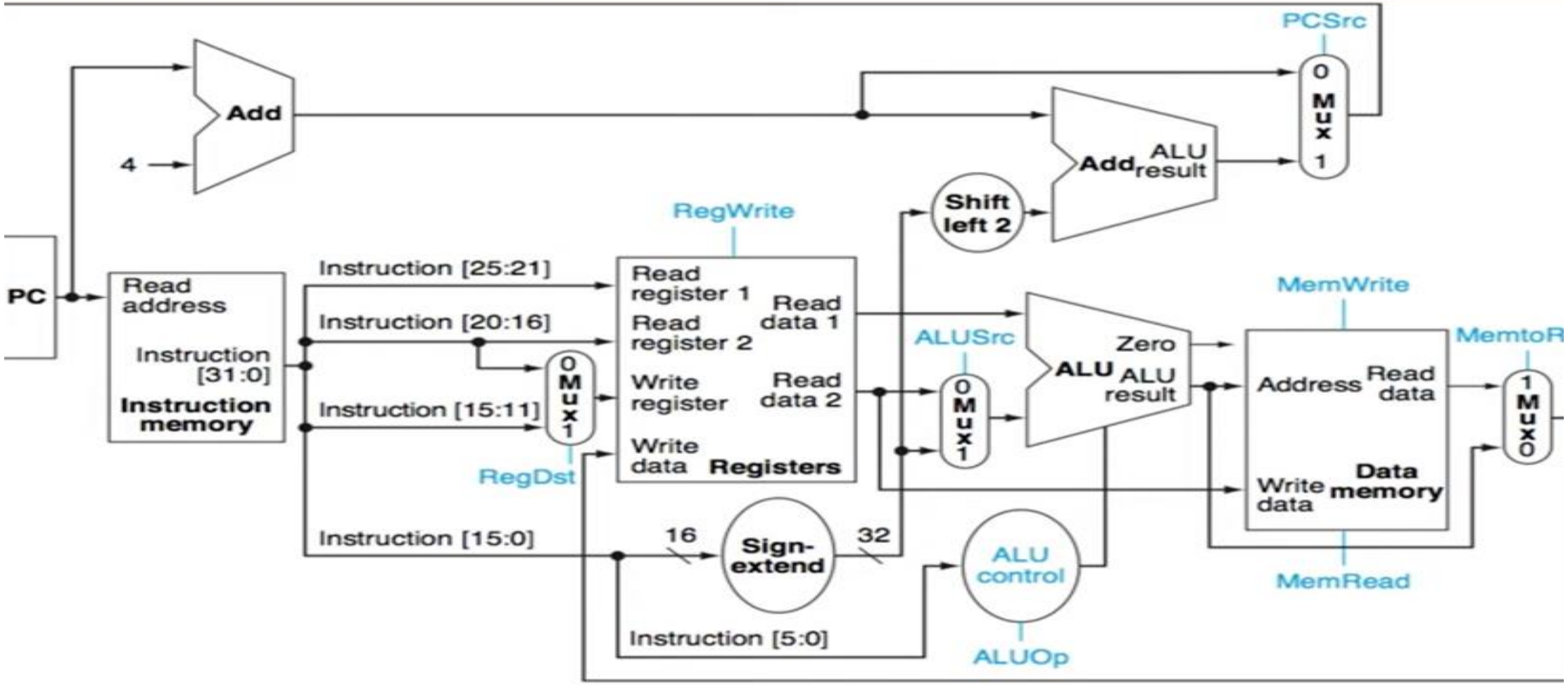
State	S	R	Q	Q'	Description
Set	1	0	0	1	Set Q'>>1
	1	1	0	1	No change
Reset	0	1	1	0	Reset Q'>>0
	1	1	1	0	No change
Invalid	0	0	1	1	Invalid Condition

Sequential Circuit Examples

Registers



Complete Data Path



Thank You.